
RNNs, LSTMs & Transformers

BSAD 8310: Business Forecasting — Lecture 9

Department of Economics

University of Nebraska at Omaha

August 24, 2026

Lecture Outline

- 1 Recurrent Neural Networks
- 2 LSTM: Long Short-Term Memory
- 3 Attention and Transformers
- 4 Key Takeaways

Recurrent Neural Networks

Process sequences step-by-step, maintaining a hidden state that carries information across time.

Why Recurrent Networks?

Limitations of FFN and CNN:

- FFN: each row processed independently — temporal order ignored unless encoded as lag features
- 1D CNN: local pattern detection within a fixed window — long-range dependencies require many stacked layers

What an RNN adds:

- A **hidden state** $\mathbf{h}_t$ passed from step to step
- Updates memory based on new input *and* prior memory
- Theoretically encodes entire history $x_1, \dots, x_t$

Vanilla RNN

$\mathbf{h}_t$ is the model's running summary up to step t . At each step:

$$\mathbf{h}_t = \tanh(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t + \mathbf{b})$$

$$\hat{y}_t = \mathbf{W}_y \mathbf{h}_t + \mathbf{b}_y$$

Same weights used at every step.

Vanishing gradient: gradients shrink exponentially during BPTT. Vanilla RNNs forget after ~ 10 steps.

LSTM: Long Short-Term Memory

Gates control what to remember, what to forget, and what to output — solving the vanishing gradient problem.

LSTM: Three Gates and Cell State

LSTM maintains: **cell state** $\mathbf{c}_t$ (long-term) and **hidden state** $\mathbf{h}_t$ (output).

Four operations per step:

1. **Forget gate:** $\mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f)$
What fraction of old cell state to keep?
2. **Input gate:** $\mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i)$
What new information to write to memory?
3. **Cell update:** $\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c)$;
 $\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$
4. **Output gate:** $\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$
What part of memory to expose as output?

Why LSTM solves vanishing gradients: $\mathbf{c}_t$ is updated additively via the forget gate. Gradients flow through additive connections without shrinking — like residual connections in ResNet.

Business analogy

Forget gate: “3-year-old data is irrelevant; forget it.” Input gate: “This holiday spike matters; remember it.”

LSTM Regressor

```
class LSTMForecaster(nn.Module):  
    def __init__(self, in_dim, hidden=128,  
                  layers=2):  
        super().__init__()  
        self.lstm = nn.LSTM(in_dim, hidden,  
                             num_layers=layers, batch_first=True,  
                             dropout=0.2)  
        self.fc = nn.Linear(hidden, 1)  
    def forward(self, x):  
        # x: (batch, seq_len, in_dim)  
        out, (h, c) = self.lstm(x)  
        return self.fc(h[-1]).squeeze(-1)
```

Key arguments:

- batch_first=True** Input: (batch, seq_len, features)
- num_layers** Stacked LSTMs (2–3 common)
- dropout** Applied between LSTM layers
- h[-1]** Last hidden state — encodes full sequence

Typical result: LSTM RMSE $\approx$ 1,400–1,600 with sequence length 24 (2 years of history).

Attention and Transformers

Instead of reading sequences step-by-step, attention lets the model look at all positions simultaneously and decide which ones matter.

The Attention Mechanism

Limitation of LSTMs: even with gating, information from 50+ steps ago may be lost or diluted.

Attention idea: let the model attend to *any* input step, weighted by relevance.

Scaled Dot-Product Attention (Vaswani et al. 2017)

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

Q = queries, K = keys, V = values; d_k = key dimension.

Business analogy

Predicting December sales: high attention to last December (lag-12) and preceding November; low weight to other months. Attention scores are *learned*, not hard-coded.

Attention computes a **weighted average** of values V , where weights reflect how well queries Q match keys K .

The Transformer Architecture

Transformer encoder block components:

1. **Multi-head self-attention:** h heads in parallel;
project: $\text{MultiHead}(Q, K, V) = \text{concat}(\text{head}_1, \dots, \text{head}_h)W^O$
2. **Residual + LayerNorm:**
 $z = \text{LayerNorm}(x + \text{MultiHead}(x))$
3. **Feed-forward sublayer:** position-wise FFN applied independently.
4. **Positional encoding:** sinusoidal embeddings encode order (attention is permutation-invariant).

Why Transformers dominate NLP:

- All positions processed in parallel (no sequential bottleneck)
- Explicit long-range dependencies via attention
- Scales to billions of parameters (BERT, GPT)

BERT and GPT

BERT (Devlin et al. 2019) (encoder-only) for classification; GPT (decoder-only) for generation. Both pre-trained on massive text corpora.

Minimal Transformer Encoder

```
enc_layer = nn.TransformerEncoderLayer(
    d_model=64, nhead=4,
    dim_feedforward=256,
    dropout=0.1, batch_first=True)
encoder = nn.TransformerEncoder(
    enc_layer, num_layers=2)
self.proj = nn.Linear(in_dim, 64)
self.enc = encoder
self.fc = nn.Linear(64, 1)
def forward(self, x):
    h = self.enc(self.proj(x))
    return self.fc(h[:, -1]).squeeze(-1)
```

Practical notes:

- Add **positional encoding** so the model knows temporal order
- Use **causal masking** to block future positions during training
- Outperforms LSTMs on longer sequences ($T > 100$)

Transformers need more data than LSTMs. For short monthly series ($T < 200$), LSTM is often a better starting point.

Key Takeaways

RNNs, LSTMs, and Transformers are the core sequence architectures in modern deep learning.

Sequence Architecture Comparison

Architecture	Long-range	Parallel	Data need	Best for
Vanilla RNN	Poor	No	Low	Very short sequences
LSTM	Good	No	Medium	Monthly/quarterly TS
1D CNN	Medium	Yes	Medium	Short TS, fast training
Transformer	Excellent	Yes	High	Long sequences, NLP, multivariate

Practical recommendation:

- Short monthly series ($T < 200$, univariate): start with LSTM
- Long series or many features: try Transformer
- Fast prototyping: 1D CNN is quickest to train
- Production at scale: pre-trained Transformer (e.g., TimesFM, Chronos)

Lecture 9: Key Takeaways

1. **RNNs** process sequences step-by-step, passing a hidden state $\mathbf{h}_t$ across time steps. Vanilla RNNs suffer from the vanishing gradient problem and forget information after ~ 10 steps.
2. **LSTMs** add a cell state $\mathbf{c}_t$ and three gates (forget, input, output) that control information flow. The additive cell update stabilizes gradients over long sequences.
3. **Attention** computes a weighted average over all positions, where weights reflect query-key similarity. It gives direct access to any past time step.
4. **Transformers** stack multi-head self-attention with residual connections and feed-forward layers. All positions are processed in parallel — no sequential bottleneck.
5. In PyTorch: use `nn.LSTM` for sequence models and `nn.TransformerEncoder` for Transformer encoders. Always use `batch_first=True` for time series data.

Next: Bayesian Statistics I — foundations of probabilistic inference for business forecasting.

References I

-  Devlin, Jacob et al. (2019). “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”. In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics*, pp. 4171–4186.
-  Vaswani, Ashish et al. (2017). “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems*. Vol. 30, pp. 5998–6008.